Mechanical and Aerospace Engineering
Machine Learning in Fluid Flow
Dr. Qiong Liu

Build and Analyze Your Own Autoencoder

Luis C. Diaz Dominguez
800772032
luisper2@nmsu.edu
October 26th, 2025

Contents

Modify the Provided Example

Build with a Different Dataset

For this homework, the selected dataset is `Fashion-MNIST`, available through `torchvision.datasets`. Two sets were downloaded: training and validation.

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, ), (0.5, ))
    ])

train_data = datasets.FashionMNIST(root = './data', train=True,
    download=True, transform = transform)
test_data = datasets.FashionMNIST(root = './data', train=False,
    download=True, transform = transform)

train_data = DataLoader(train_data, batch_size = 128, shuffle = True)
test_data  = DataLoader(test_data,  batch_size = 128, shuffle = False)
```

Network Architecture

To accomplish this task, the original code, which used dense layers to predict the output, was modified. In this version, the number of neurons between the encoder and decoder is dynamic. Additionally, batch normalization is applied between each layer in the encoder.

```python
import torch, torch.nn as nn, torch.nn.functional as F

class Autoencoder(nn.Module):
    def __init__(self, dims = 32):
        super().__init__()

        ''' Encoder '''
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 32, 3, stride = 2, padding = 1),
            nn.BatchNorm2d(32),
            nn.ReLU(True),
            nn.Conv2d(32, 64, 3, stride = 2, padding = 1),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.Dropout(0.2)
        )

        self.fc_mu  = nn.Linear(64 * 7 * 7, dims)
        self.fc_dec = nn.Linear(dims, 64 * 7 * 7)

        ''' Decoder '''
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(64, 32, 3, stride = 2, padding = 1,
                output_padding = 1),
            nn.ReLU(True),
            nn.ConvTranspose2d(32, 1, 3, stride = 2, padding = 1,
                output_padding = 1),
            nn.Tanh()
        )

    def forward(self, x):
        # Encode
        x = self.encoder(x)
        x = x.view(x.size(0), -1)
        x = self.fc_mu(x)

        # Decode
        x = self.fc_dec(x)
        x = x.view(x.size(0), 64, 7, 7)
        x = self.decoder(x)

        return x
```

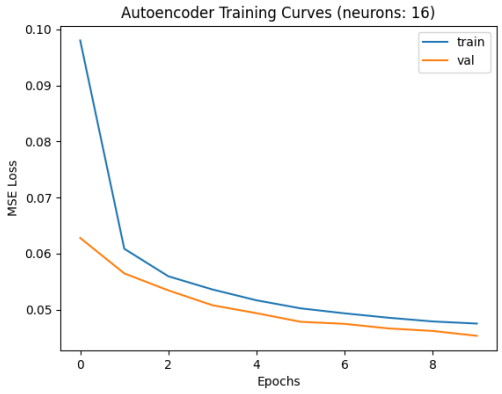
Latent Space Dimensions

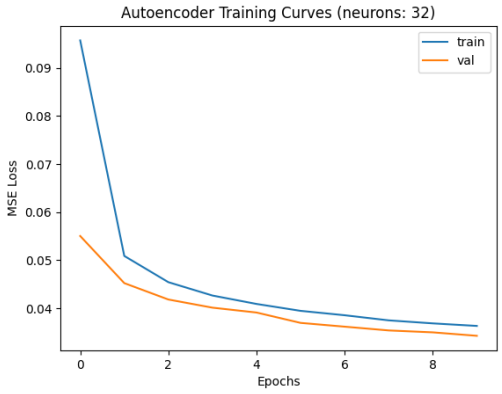
The chosen latent space dimensions for analysis were 16, 32, 64, and 128 neurons. These values were used to compare how the number of latent features affects model performance and reconstruction accuracy.

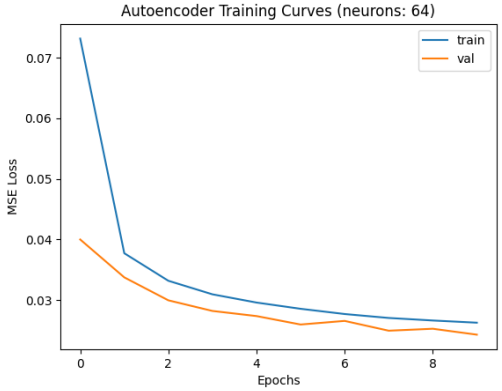
```
dimensions = 128
model = Autoencoder(dimensions)

criteria = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr = 0.001)

epochs = 10
training_loss, val_loss = [], []

for epoch in range(epochs):
    model.train()
    loss_cum = 0

    for x, _ in train_data:
        x = x.to()
        recon = model(x)
        loss = criteria(recon, x)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        loss_cum += loss.item()

    training_loss.append(loss_cum / len(train_data))

    model.eval()
    val_cum = 0

    with torch.no_grad():
        for x, _ in test_data:
            x = x.to()
            val_cum += criteria(model(x), x).item()

    val_loss.append(val_cum / len(test_data))
    print(f'Epoch {epoch + 1}/{epochs} - Train:{(loss_cum /
        len(train_data)):4f} - Val:{(val_cum / len(test_data)):4f}')
```

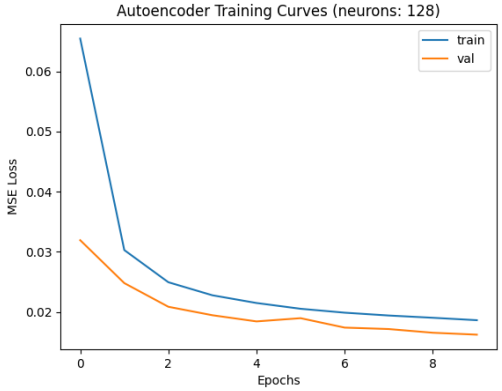
Visualization and Analysis

Training Curves

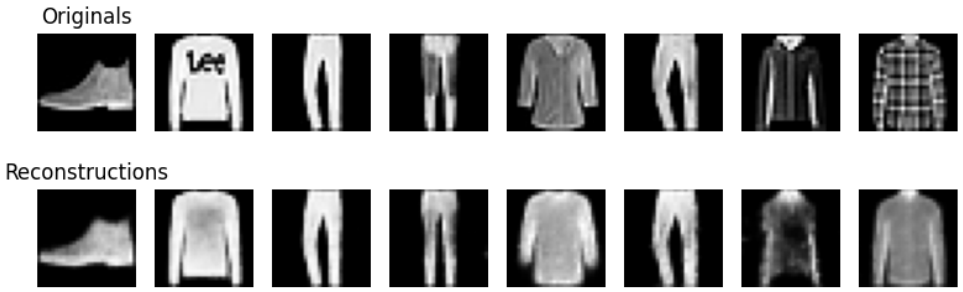




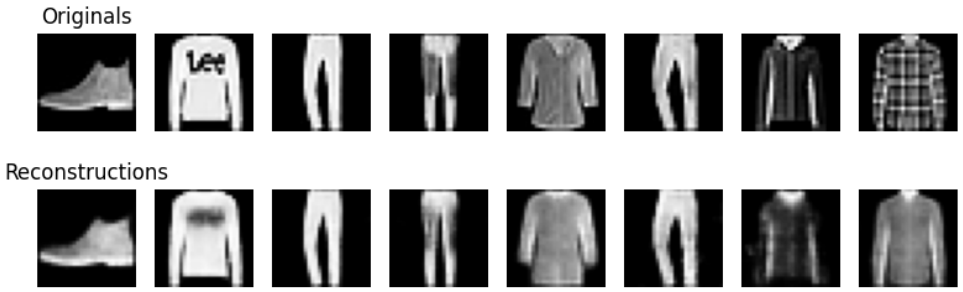




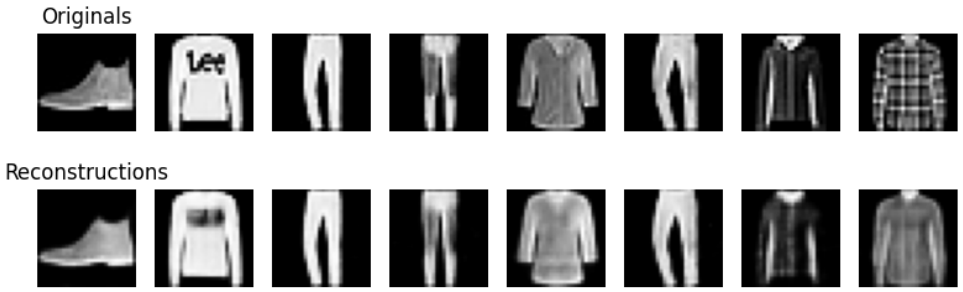
Reconstruction Results



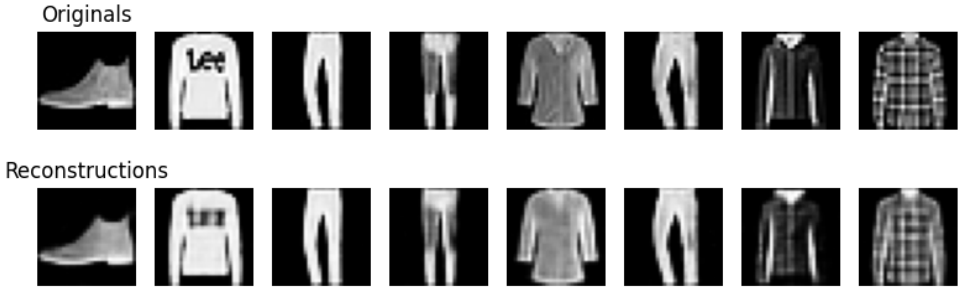
(a) 16 Neurons



(b) 32 Neurons



(c) 64 Neurons



(d) 128 Neurons

Discussion

I noticed that while the training performance slightly decreased, the quality of the predictions improved. The model achieved higher accuracy in reconstructing fine details as the number of neurons increased, but overall, the performance remained relatively consistent. Additionally, I learned that plotting the neuron activations helps visualize how the network learns and represents patterns in the data.

Reflection and Report

During this assignment, I modified the encoder stage according to the homework requirements. I also made the architecture dynamic, allowing the number of neurons to change. I observed that the reconstruction quality improved with more neurons, but the method became slower than the original version. A possible improvement would be to make the training process more efficient. Overall, this project helped me understand how convolutional autoencoders learn spatial features and how the latent space dimension directly influences both reconstruction quality and computational cost.